

ENGENHARIA DE SOFTWARE

UNIDADE 1 - AULA 3

Gilberto Falco Netto

Tecnólogo em Análise e Desenvolvimento de Sistemas,
Faculdade Anhanguera - Jundiaí/SP

Sumário

- 1** Visão Geral
- 2** Fundamentos Detalhados do
Controle de Versões
- 3** Gerenciamento de
Configuração de Software
(GCS) Avançado

A Necessidade Crítica do Controle de Versões

- Software: um organismo vivo em constante evolução.
- Desenvolvimento colaborativo: orquestrando a sinfonia de múltiplos colaboradores.
- Histórico de alterações: a bússola para rastreabilidade e auditoria.
- Reversão de mudanças: o botão "desfazer" para a correção de erros e recuperação de versões.

Conceitos Fundamentais: Versões, Revisões, Commits e Metadados

- Versão: o software congelado no tempo, um instantâneo de seu estado.
- Revisão: a mudança incremental, a pincelada que transforma a versão.
- Commit: o registro imutável da revisão, a cápsula do tempo da mudança.
- Metadados: a história por trás da mudança, autor, data e mensagem de commit.

Repositório: O Cofre do Tesouro do Software

- Local de armazenamento centralizado: o cofre que guarda arquivos e histórico.
- Controle de acesso: o guardião das permissões e da segurança.
- Organização do projeto: a arquitetura que estrutura diretórios e arquivos.
- Histórico completo: a linha do tempo que revela todas as versões e revisões.

Fluxo de Trabalho Detalhado: Do Checkout ao Merge

- ❶ Checkout/Clone: a jornada começa com a obtenção da versão mais recente.
- ❷ Alterações locais: a criatividade flui, os arquivos se transformam.
- ❸ Commit: a mudança é registrada, a cápsula do tempo é selada.
- ❹ Push: a mudança viaja para o repositório, compartilhando a criação.
- ❺ Pull/Merge: a colaboração se encontra, as mudanças se integram.

Tipos de Controle de Versões: Do Local ao Distribuído

- Local: o controle solitário, a versão única na máquina.
- Centralizado: o repositório central, os clientes locais em harmonia.
- Distribuído: a autonomia reina, cada cliente com a cópia completa.

Vantagens do Controle de Versões Distribuído: A Liberdade e a Segurança

- Trabalho offline: a produtividade não conhece fronteiras, commits locais sem conexão.
- Segurança: a redundância protege, dados em cada cópia do repositório.
- Flexibilidade: a adaptabilidade se destaca, suporte a diversos fluxos de trabalho.
- Desempenho: a velocidade impressiona, operações locais instantâneas.

GCS: A Orquestração da Evolução do Software

- Definição: o maestro que rege as mudanças no software.
- Objetivos: a sinfonia da organização, rastreabilidade, qualidade e eficiência.
- Componentes: a orquestra de pessoas, processos, ferramentas e documentação.

Atividades do GCS: Identificação de Configuração - O Inventário do Software

- Identificar itens de configuração: o inventário completo de arquivos e documentos.
- Esquemas de identificação: a catalogação precisa de nomes e versões.
- Ferramentas e técnicas: o arsenal para gerenciar cada item.

Atividades do GCS: Controle de Mudanças - A Gestão da Transformação

- Procedimentos: o protocolo para solicitar, aprovar e implementar mudanças.
- Controle de acesso: a definição de quem pode fazer o quê.
- Rastreamento de mudanças: o histórico completo da transformação.
- Gestão de conflitos: a resolução harmoniosa de mudanças conflitantes.

Atividades do GCS: Auditoria e Relatórios - A Prestação de Contas

- Auditoria: a verificação da conformidade com os padrões de GCS.
- Relatórios: a geração de informações sobre o estado do software.
- Métricas: a medição da eficácia do processo de GCS.

Mitos e Verdades do GCS: Desvendando a Complexidade

Mito: GCS é burocrático e lento.

Verdade: GCS automatiza tarefas e acelera a eficiência.

Mito: GCS é só para grandes projetos.

Verdade: GCS é valioso para projetos de todos os tamanhos, como um pequeno jardim que necessita de cuidados.

Implementando o GCS: O Guia para o Sucesso

- Plano de GCS: o mapa claro e conciso para a jornada.
- Ferramentas certas: o arsenal adequado para cada desafio.
- Treinamento da equipe: a capacitação para dominar as práticas de GCS.
- Monitoramento e ajustes: a busca contínua pela melhoria.
- Automação: a delegação de tarefas repetitivas para a eficiência.

Baselines e Releases: Os Marcos da Evolução

- Baseline: o estado aprovado, a base sólida para o futuro.
- Release: a versão entregue, o fruto do trabalho da equipe.
- Tags: os marcadores que identificam baselines e releases.

Branches e Merges: A Arte do Desenvolvimento Paralelo

- Branch: a linha de desenvolvimento paralela, a exploração em novas direções.
- Merge: a integração harmoniosa, a união de diferentes caminhos.
- Estratégias de branching: a escolha do modelo ideal para o projeto.
- Resolução de conflitos: a superação dos obstáculos na integração.

Comandos Essenciais do Git

- `git init`: o nascimento do repositório, a criação do espaço de controle.
- `git clone`: a clonagem do repositório, a cópia fiel para a máquina local.
- `git add`: a preparação da mudança, a adição de arquivos ao palco do commit.
- `git commit`: o registro da mudança, a cápsula do tempo da transformação.

Comandos Essenciais do Git

- `git push`: a viagem da mudança, o envio para o repositório remoto.
- `git pull`: a integração da mudança, a atualização do repositório local.
- `git branch`: a criação do caminho paralelo, o branch para a exploração.
- `git merge`: a união dos caminhos, a integração dos branches.

GitHub: A Plataforma de Colaboração e Hospedagem

- Hospedagem de repositórios: o lar dos projetos, o espaço para a colaboração.
- Ferramentas de colaboração: o palco para a troca de ideias, pull requests e issues.
- Comunidade global: o encontro de mentes criativas, o berço de projetos open source.
- Pull Requests: a revisão colaborativa, o aperfeiçoamento do código em equipe.
- Issues: o rastreamento de desafios, a organização de tarefas e discussões.
- GitHub Actions: a automação integrada, a otimização de fluxos de trabalho.

Fluxo de Trabalho Detalhado: Da Criação do Branch ao Merge Final

- Criar um repositório: o início da jornada, o espaço para o projeto.
- Clonar o repositório: a cópia fiel, a base para o trabalho local.
- Criar um branch: o caminho paralelo, a exploração de novas ideias.
- Alterações e commits: a transformação do código, a jornada da criação.

Fluxo de Trabalho Detalhado: Da Criação do Branch ao Merge Final

- Push do branch: a viagem da mudança, o compartilhamento com a equipe.
- Pull Request: a revisão colaborativa, o aperfeiçoamento do código.
- Discussão e revisão: a troca de ideias, o aprimoramento do projeto.
- Merge: a união dos caminhos, a integração das mudanças.
- Resolução de conflitos: a superação dos obstáculos, a harmonia na integração.
- Fechar o PR: a conclusão da tarefa, o fechamento do ciclo.

Recursos Avançados do Git: Stashing e Rebasing - A Arte da Eficiência

- Stashing: o armazenamento temporário, a organização do trabalho em progresso.
- Rebasing: a reescrita do histórico, a busca pela linearidade e clareza.
- Vantagens do rebasing: a história linear, a redução de conflitos.
- Riscos do rebasing: a reescrita do passado, o cuidado em branches compartilhados.

Recursos Avançados do Git: Cherry-picking e Submodules - A Precisão e a Modularidade

- Cherry-picking: a seleção precisa, a aplicação de commits específicos.
- Submodules: a modularidade em ação, a inclusão de outros repositórios.
- Casos de uso: bibliotecas compartilhadas, projetos modulares.

GitHub Actions: A Automação Inteligente - O Futuro do Desenvolvimento

- Automação de tarefas: a delegação de trabalho, testes, builds e deploys.
- Fluxos de trabalho: a definição de tarefas, a orquestração da automação.
- Integração: a sinergia com o ecossistema GitHub, a conexão com o futuro.
- Exemplos: CI/CD, análise de código, automação de issues.

Estratégias de Branching: Gitflow e GitHub Flow - A Escolha do Caminho

- Gitflow: o modelo complexo, a gestão de releases e hotfixes.
- GitHub Flow: o modelo simplificado, o deploy direto do branch principal.
- Escolhendo a estratégia: a decisão sob medida para o projeto.

CVS: Sistema de Controle de Versões Centralizado

- O Legado da Centralização

- Sistema centralizado: o repositório único, a coordenação centralizada.
- Menos flexível: a rigidez do modelo, a limitação da autonomia.
- Comandos básicos: checkout, commit e update, a simplicidade na operação.
- Repositório CVS: a estrutura de diretórios, a organização do histórico.

Operações Básicas do CVS: A Simplicidade na Centralização

- Checkout: a obtenção da cópia, a jornada de trabalho.
- Commit: o envio da mudança, a atualização do repositório.
- Update: a integração da mudança, a sincronização da cópia.
- Merge: a mescla das mudanças, a união dos trabalhos.

Terminologia do CVS: A Linguagem da Centralização

- Módulo: a hierarquia de diretórios, a organização do projeto.
- Release: o lançamento da versão, a entrega do produto.
- Export: a obtenção sem controle, a cópia sem histórico.
- Import: a criação do módulo, a adição ao repositório.